

Grundlagen der C-Programmierung

Dr. Henrik Brosenne
Georg-August Universität Göttingen
Institut für Informatik

Wintersemester 2024/25

Funktionen

Vereinbarung von Funktionen

Funktionswerte

Aufruf von Funktionen

Parameter und Argumente

Zeiger als Parameter

Felder als Parameter

Das Attribut `const`

Rekursion

Einschub: Das Modulkonzept

Vereinbarung von Funktionen

Die Vereinbarung einer Funktion besteht in C aus folgenden Teilen.

- In einer **Deklaration** werden die Eigenschaften einer Funktion bekanntgemacht.
Eigenschaften sind Name, Parameterliste und Typ des Funktionswerts.
- In der **Definition** werden die Operationen festgelegt, Speicherplatz reserviert, usw.

Es ist offensichtlich, dass die Definition einer Funktion ihrer Deklaration entsprechen muss.

Eine **Funktionsdeklaration** hat die Form

```
typ name (type-1 [parameter-1], ..., type-N [parameter-N]);
```

Ein Funktionsdeklaration wird als **Prototyp** bezeichnet.

Seit C99 muss der Typ des Funktionswerts angegeben werden. Bei ANSI-C wurde bei fehlenden Typ des Funktionswerts implizit `int` ergänzt.

Deklaration von Funktionen

Für Funktionsnamen gelten dieselben Regeln wie für Namen von Variablen und Konstanten.

Wenn eine Funktion keine Parameter besitzt, **kann** dieses durch das Schlüsselwort **void** gekennzeichnet werden.

Beispiel

Eine Funktion soll die Potenz x^y einer double-Zahl x und einer ganzen Zahl y berechnet ($y \geq 0$).

Der Prototyp kann dann die folgende Form haben.

```
double power (double basis, int exponent);
```

oder

```
double power (double, int);
```

Obwohl auch die zweite Deklaration den Intentionen C vollständig entspricht, ist doch die erste Deklaration vorzuziehen, weil sie nicht nur die Typen der Parameter, sondern durch die Wahl der Namen auch deren Bedeutung beschreibt.

Definition von Funktionen

In der Definition einer Funktion werden die Operationen festgelegt, die beim Aufruf der Funktion durchzuführen sind.

Die Funktionsdefinition hat die Form

```
type name (type-1 parameter-1, ..., type-N parameter-N) {  
    (local) declarations  
    statements  
}
```

Achtung. In der Definition einer Funktion wird der **Funktionsheader** **nicht** wie in der Deklaration durch ein Semikolon abgeschlossen.

Beispiel 1

Potenzieren

```
1  double power(double basis, int exponent);
2
3  // ...
4
5  double power(double basis, int exponent) {
6      double p = 1.0;
7
8      while (exponent--)
9          p *= basis;
10
11     return p;
12 }
```

Beispiel 2

Eine Funktion `string_length` soll die Länge eines String bestimmt, ohne das abschließende Null-Zeichen mitzuzählen.

```
1  int string_length(char s[]);
2
3  // ...
4
5  int string_length(char s[]) {
6      int i = 0;
7      while(s[i])
8          i++;
9      return i;
10 }
```

Die Länge des übergebenen Strings ergibt sich aus seinem Inhalt. Das Null-Zeichen zeigt das Ende an.

Im allgemeinen muss bei Feldern als Funktionsparameter die Länge des Feldes übermittelt werden, z.B. durch einen separaten Parameter.

Prototypen

Prototypen, d.h. Funktionsdeklarationen mit Angabe der Parameterliste, haben verschiedene Zwecke.

- Der Compiler kann prüfen, ob Argumente und Parameter einer Funktion übereinstimmen oder doch zumindest zueinander „passen“, ohne den für die Funktion auszuführenden Code zu kennen.

Das ist häufig dann der Fall, wenn die Funktion aus einer Bibliothek stammt, deren Quelltext nicht veröffentlicht wurde.

- Der Compiler kann, soweit erforderlich und möglich, die Typen der Argumente in die Typen der Parameter umwandeln. Die Umwandlung erfolgt nach denselben Regeln wie bei Wertzuweisungen.
- Der Compiler kann einen Funktionswert bei Bedarf in einen anderen Typ umwandeln.

Damit die Prüfungen und ggf. Umwandlungen ausgeführt werden können, muss der Prototyp einer Funktion in der Quelldatei **vor** ihrem ersten Aufruf stehen. Die Definition kann dann an beliebiger anderer Stelle folgen.

Grundlegenden Formalien

- Eine Funktionsdefinition darf nicht innerhalb einer anderen Funktionsdefinition stehen, d.h. C erlaubt keine Schachtelung von Funktionsdefinitionen. Für Funktionsaufrufe gilt das nicht, wie wir bereits gesehen haben: Funktionen können ohne weiteres andere Funktionen aufrufen.
- Deklarationen, die innerhalb eines Funktionsrumpfes vereinbart werden, (z.B. in den beiden Beispielen jeweils die Variable `i`) sind in der Regel lokal, d.h. sie sind nur innerhalb der jeweiligen Funktion bekannt.

Funktionen

Vereinbarung von Funktionen

Funktionswerte

Aufruf von Funktionen

Parameter und Argumente

Zeiger als Parameter

Felder als Parameter

Das Attribut `const`

Rekursion

Einschub: Das Modulkonzept

Funktionswerte

Welchen **Funktionswert** eine Funktion an die rufende Funktion zurückliefert, wird durch die **return**-Anweisung

```
return [expression];
```

festgelegt. Stimmt der Typ des Ausdrucks *ausdruck* nicht mit dem Typ der Funktion überein, erfolgt eine Typumwandlung wie bei einer Wertzuweisung.

Eine Funktion braucht keinen Funktionswert zurückzuliefern.

Dieses kennzeichnet man in Deklaration und Definition, indem man als Typ des Funktionswertes das Schlüsselwort **void** angibt.

Außerdem entfällt in der **return**-Anweisung der Ausdruck; eine **return**-Anweisung ohne Ausdruck direkt vor der schließenden Klammer des Funktionsrumpfes kann man sogar ganz weglassen, da die Kontrolle bei ihrem Erreichen auf jeden Fall an die aufrufende Funktion zurückgeht.

Beispiel

```
void do_nothing() {  
    return;  
}
```

Funktionen

Vereinbarung von Funktionen

Funktionswerte

Aufruf von Funktionen

Parameter und Argumente

Zeiger als Parameter

Felder als Parameter

Das Attribut `const`

Rekursion

Einschub: Das Modulkonzept

Aufruf von Funktionen

Der **Aufruf** einer Funktion besteht aus dem Namen der Funktion, dem, in runde Klammern eingeschlossen, die Liste der **Funktionsargumente** folgt.

Die Funktionsargumente sind die Werte, die die Funktion während ihrer Ausführung benutzen soll.

Beispiel

```
i = power(2.0, 5);  
j = string_length("How long is this string?");  
do_nothing();
```

Durch ihren Aufruf geht die Kontrolle auf die Funktion über, bis diese eine weitere Funktion aufruft oder die Kontrolle an die rufende Funktion zurückgibt.

Zulässig, wenn auch sinnlos, wären ebenso die Aufrufe

```
power(2.0, 5);  
string_length("How long is this string?");
```

Nicht zulässig ist

```
z = do_nothing();
```

Funktionen

Vereinbarung von Funktionen

Funktionswerte

Aufruf von Funktionen

Parameter und Argumente

Zeiger als Parameter

Felder als Parameter

Das Attribut `const`

Rekursion

Einschub: Das Modulkonzept

Parameter und Argumente

Kommunikation zwischen rufender und gerufener Funktion:

In der Definition einer Funktion können (**formale**) **Parameter** definiert werden.

Die Parameter stehen innerhalb der Funktion wie (lokale) Variablen des entsprechenden Typs zur Verfügung.

Die Werte der Parameter werden beim Aufruf der Funktion durch die **Argumente** (**Aktualargumente**) festgelegt.

Die Argumente werden den Parametern linear zugeordnet; die einzelnen Zuordnungen erfolgen wie **Wertzuweisungen**.

Die Anzahl der Argumente muss mit der Anzahl der Parameter übereinstimmen und die Typen der Argumente müssen mit den Typen der Parameter übereinstimmen.

Die Argumente werden ggf. wie bei einer Wertzuweisung in die Typen der Parameter umgewandelt werden. Diese impliziten Typumwandlungen werden vom Standard vorgeschrieben.

Parameter und Argumente

Prototyp der Funktion **power**

```
double power(double basis, int exponent);
```

und ihren Aufruf

```
power(a, j)
```

Die Funktion **power** hat die Parameter **basis** und **exponent**. Beim Aufruf wird **basis** der Wert von **a** und **exponent** der Wert von **j** zugewiesen.

Die Parameter einer Funktion entsprechen lokalen Variablen des jeweiligen Typs, in die beim Aufruf die Werte der Argumente **kopiert** werden.

Dieser Ablauf wird als **call by value** bezeichnet.

Konsequenzen. Eine Veränderung eines Parameters innerhalb der Funktion hat **keinerlei** Auswirkungen auf die Umwelt der Funktion.

call by value, Beispiel (1/2)

```
1  double power(double basis, int exponent) {
2      double p = 1.0;
3
4      if (exponent < 0)
5          exponent = -exponent, basis = 1.0/basis;
6
7      while (exponent-->0)
8          p *= basis;
9
10     return p;
11 }
```

Die Funktion verändert die Parameter `basis` und `exponent`. Das hat, wenn die Funktion durch

```
double a = 2.0, x;
int      j = 5;
x = power (a, j);
```

aufgerufen wird, aber keinerlei Folgen für die Variablen `a` und `j` der rufenden Funktion, weil ja nur **Kopien** der Werte von `a` und `j` an `power` übergeben werden.

call by value, Beispiel (2/2)

```
1 double power(double basis, int exponent) {
2     double p = 1.0;
3
4     if (exponent < 0)
5         exponent = -exponent, basis = 1.0/basis;
6
7     while (exponent-->0)
8         p *= basis;
9
10    return p;
11 }
```

Der Aufruf

```
double x;
x = power(2.0, 5);
```

ist nur deshalb zulässig, weil die Funktion nicht mit den Argumenten selbst arbeitet, sondern weil die Parameter sich wie lokalen Variablen verhalten, in die die Werte der Argumente **kopiert** werden.

Die konstanten Argumenten haben keine Adressen und könnten nicht verändert werden.

Übergabe von Adressen

Nicht in jedem Falle ist es zweckmäßig und wünschenswert, dass Funktionen ihre Parameter nicht verändern können (Beispiel `scanf`).

In solchen Fällen müssen die **Adressen** der Argumente anstelle ihrer **Werte** übergeben werden. Das kann man z.B. mit dem Adressoperator `&` erreichen.

Übergabe von Adressen bezeichnet man allgemein als **call by reference**.

C kennt formal nur **call by value**.

Hintergrund

```
scanf ("%d", &i)
```

der Compiler betrachtet `&i` nicht als **Adresse der Variablen i**, sondern als **Ausdruck mit einem Zeigertyp** und übergeben wird **der Wert** dieses Ausdrucks.

Es reicht natürlich **nicht** aus, einen Zeiger an eine Funktion zu übergeben, um ihr die Veränderung ihrer Parameter zu ermöglichen. Auch die Vereinbarung der Parameter und der Zugriff auf sie müssen entsprechend formuliert sein.

Funktionen

Vereinbarung von Funktionen

Funktionswerte

Aufruf von Funktionen

Parameter und Argumente

Zeiger als Parameter

Felder als Parameter

Das Attribut `const`

Rekursion

Einschub: Das Modulkonzept

Zeiger als Parameter

Wenn Zeiger an eine Funktion übergeben werden sollen, müssen Deklaration und Definition passend formuliert werden.

- Im Funktionsheader muss markiert werden, dass ein Parameter ein Zeiger ist. Das geschieht, indem dem Namen des Parameters ein Stern (*) vorangestellt wird.
- Im Funktionsrumpf ist der Zeiger (die Adresse) eine Kopie des übergebenen Zeigers.

Der Zeiger verweist aber nach wie vor auf eine Speicherstelle, deren Inhalt mit dem dereferenzierten Zeiger gelesen/geschrieben werden kann.

Beispiel

Eine Funktion soll den Wert einer Variablen der rufenden Funktion um 1 erhöhen.

```
1 void increment_malfunction(int i) {  
2     i = i + 1;  
3 }
```

Die Funktion erhält eine **Kopie** des Wertes von `i`, da nur diese Kopie verändert wird hat das keine Auswirkungen auf die Variablen der rufenden Funktion.

Korrigiertes Beispiel

```
1 void increment(int *i) {  
2     *i = *i + 1;  
3 }
```

`i` ist eine **Zeigervariable**, festgelegt durch die Parametervereinbarung `int *i`.

Die (automatische) Dereferenzierung von `i` im Zuge der Auswertung eines Ausdrucks liefert den Wert von `i`, einen **Zeiger** auf eine `int`-Variable.

Soll auf den `int`-Wert zugegriffen werden, muss also **noch einmal** explizit dereferenziert werden, mit `*i`.

Funktionsaufruf

```
int j = 1;  
increment(&j);
```

`j` hat nach dem Aufruf von `increment` den Wert 2.

Funktionen

Vereinbarung von Funktionen

Funktionswerte

Aufruf von Funktionen

Parameter und Argumente

Zeiger als Parameter

Felder als Parameter

Das Attribut `const`

Rekursion

Einschub: Das Modulkonzept

Felder als Parameter

Felder nehmen als Parameter (und Argumente) eine Sonderstellung ein.

Der Name eines Feldes ist eine Zeigerkonstante, deren Wert die Adresse der ersten Komponente des Feldes ist.

Die Konsequenz ist, dass bei Feldern keine Kopie der Werte der Komponenten an eine Funktion übergeben wird, sondern stets nur die (Anfangs-)Adresse des Feldes.

Sieht man bei einer Funktion ein Feld als Parameter vor, so muss man darauf achten, dass man nicht unerwünschte Nebeneffekte durch die Veränderung von Feldkomponenten erzeugt.

Vereinbaren

Es gibt mehrere Wege Funktionen mit (eindimensionalen) Feldern als Parameter zu vereinbaren.

```
int string_length(char s[]);
```

```
int string_length(char *s);
```

```
int string_length(char s[64]);
```

Alle drei Deklarationen erzeugen identischen Resultate.

Jede Deklaration sagt dem Compiler das der Parameter ein Zeiger auf `char` ist.

C führt keinen Kontrolle der Feldgrenzen durch, die Dimension in der dritten Zeile wird **ignoriert**. Aus Sicht des Compilers wird hier als Parameter ein Zeiger auf ein Feld (beliebiger Größe) vereinbart.

Invertieren eines Strings (1/3)

```
1  #include <stdio.h>
2  #define N 20
3
4  /*****
5  void reverse(char str[]);
6  int string_length(char str[]);
7
8  /*=====*/
9  int main() {
10     char str[N+1];
11     int n;
12
13     printf("String (length max. %d): ", N);
14     scanf("%s", str);
15     reverse(str);
16     printf("Reverse string: '%s'\n", str);
17
18     return 0;
19 }
20
```

Invertieren eines Strings (2/3)

```
21  /*-----*/
22  void reverse(char str[])
23  {
24      int i, j;
25      char c;
26
27      int n = string_length(str);
28      for (i = 0, j = n - 1; i < j; i++, j--)
29          c = str[i], str[i] = str[j], str[j] = c;
30  }
31
32  /*-----*/
33  int string_length(char str[])
34  {
35      int i = 0;
36      while (str[i])
37          i++;
38
39      return i;
40  }
```

Invertieren eines Strings (3/3)

Anmerkungen

- Das Rahmenprogramm „verlässt“ sich darauf, dass der Benutzer keinen zu langen String eingibt; dieser String darf keine „white spaces“ enthalten.
- Das Hauptprogramm ist wie üblich als erstes angegeben und erst danach die Funktionen.
- Die Funktionsdeklarationen von **reverse** und **string_length** stehen **vor** dem Hauptprogramm. Auch das ist üblich. Wir hätten auch **reverse** innerhalb des Hauptprogramms und **string_length** innerhalb von **reverse** deklarieren können; das hätte zur Folge gehabt, dass **reverse** nur innerhalb des Hauptprogramms und **string_length** nur innerhalb von **reverse** zur Verfügung steht.

Erinnerung. Nur Feldnamen werden als Zeigerkonstanten betrachtet; für Feldkomponenten gilt das nicht.

```
a = power(t[k], 4);
```

Der Wert von **t[k]** wird an **power** übergeben und nicht die Adresse.

Funktionen

Vereinbarung von Funktionen

Funktionswerte

Aufruf von Funktionen

Parameter und Argumente

Zeiger als Parameter

Felder als Parameter

Das Attribut `const`

Rekursion

Einschub: Das Modulkonzept

Das Attribut `const`

Eine Funktion, die ein Feld, das als Argument übergeben bekommt, hat den direkten Zugriff auf die Originaldaten und kann diese ggf. verändern.

Der Programmierer muss sehr diszipliniert arbeiten, damit nicht ungewollte und ungewünschte Veränderungen erfolgen.

Der Standard gibt dem Programmierer Unterstützung. Versieht man einen Parameter mit dem Attribut `const`, so erzeugt der Compiler bei jeder Zuweisung an den Parameter, bei Feldern auch an eine Komponente, eine Fehlermeldung.

Bei Parametern ist das Attribut `const` zunächst nur für Felder sinnvoll, weil man bei *einfachen* Variablen ohnehin nur eine Kopie des Wertes erhält, die man unbesorgt verändern kann.

Beispiel

```
int string_length(const char str[]);
```

Einsatzmöglichkeiten

- Bei vielen Funktionen, etwa den Funktionen der Standardbibliothek, kann man sich nur die Deklarationen in einer Headerdatei und ggf. eine zugehörige Beschreibung ansehen, während die Definition nicht zugänglich ist.
- Je länger eine Funktion ist, desto mühsamer wird die Prüfung, ob sie einen Parameter verändert oder nicht.

In beiden Fällen ist es angenehm, schon aus dem Prototyp ablesen zu können, ob die Funktion ein übergebenes Feld unverändert lässt.

Das Attribut `const` ist nicht auf die Verwendung mit Feldern begrenzt.

Funktionen

Vereinbarung von Funktionen

Funktionswerte

Aufruf von Funktionen

Parameter und Argumente

Zeiger als Parameter

Felder als Parameter

Das Attribut `const`

Rekursion

Einschub: Das Modulkonzept

C erlaubt **Rekursion**.

- **Direkter Rekursion**, eine Funktion ruft sich selbst auf.
- **Indirekter Rekursion**, zwei oder mehr Funktionen rufen sich wechselseitig auf.

Rekursion wird in der Mathematik häufig in Definitionen verwendet.

Klassisches Beispiel

Berechnung der Fakultät $n!$ einer nichtnegativen ganzen Zahl n .

$$n! := \begin{cases} n \cdot (n-1)! & \text{für } n > 0 \\ 1 & \text{für } n = 0 \end{cases}$$

Mit rekursiven Funktionen lassen sich solche Formeln direkt umsetzen.

```
1 long int fac(int n) {  
2     if (n < 0)                // error  
3         return 0;  
4  
5     if (n == 0)               // base case  
6         return 1;  
7  
8     return n * fac(n - 1);    // recursion  
9 }
```

Die Rekursion darf nicht beliebig lange fortgefahren werden.

Im Beispiel sorgt Folgendes dafür.

- $n < 0$ **Fehlerfall**
- $n == 0$ **Basisfall**
- $\text{fac}(n - 1)$ bei **Rekursion** Inkrement des Arguments

Iteration

Rekursiv formulierte Berechnungen müssen nicht immer rekursiv realisiert werden.

Rekursion ist in der Regel ziemlich aufwendig.

- Parameter müssen kopiert werden („call by value“).
- Lokale Variablen müssen bereitgestellt.
- Der Unterprogrammsprung muss ausgeführt werden.

Oft gibt es für eine rekursiv formulierte Aufgabe auch eine einfache Lösung mit einer Schleife (**iterative Lösung**).

Klassisches Beispiel, iterativ

Iterative Definition der Fakultätsfunktion

$$n! = \prod_{i=1}^n i$$

Quelltext

```
1  long int fac(int n) {  
2      long int result = 1;  
3  
4      if (n < 0)          // error  
5          return 0;  
6  
7      while (n > 1)  
8          result *= n--;  
9  
10     return result;  
11 }
```

Rekursion oder Iteration

Ob eine rekursive oder eine nicht–rekursive Realisierung bestimmter Funktionen vorzuziehen ist, hängt sehr vom Einzelfall ab.

Immer dann, wenn sich Rekursion, wie bei den Fakultäten, durch eine einfache Schleife oder ein ähnlich einfaches anderes Konstrukt vermeiden lässt, sollte man sie auch vermeiden.

Andererseits gibt es Fälle, in denen Rekursion nur mit erheblichen „Klimmzügen“ zu vermeiden wäre – und dann sollte man sie auch nutzen.

Funktionen

Vereinbarung von Funktionen

Funktionswerte

Aufruf von Funktionen

Parameter und Argumente

Zeiger als Parameter

Felder als Parameter

Das Attribut `const`

Rekursion

Einschub: Das Modulkonzept

Das Modulkonzept

Will man Funktionen bereitstellen, die in unterschiedlichen Programmen genutzt werden können, müssen die Funktionen und das nutzende Programm in verschiedenen Quelldateien stehen.

Wenn wir die Funktionen z.B. Funktionen für die Erkennung von Binärziffern (`isbdigit`) und die Umwandlung von Strings aus Binärziffern in Dezimalzahlen (`btoi`) in eine separate Quelldatei schreiben, etwa `bin.c`, kann diese Datei wie folgt eingefügt werden.

```
#include "bin.c"
```

Die Anführungszeichen bewirken, dass der Präprozessor an anderen Stellen im Dateibaum sucht, als wenn spitze Klammern verwendet werden.

Dieses Vorgehen erweist sich bei der praktischen Arbeit an größeren Projekten schnell als unzureichend.

Man braucht eine Möglichkeit, Teile eines Programms voneinander unabhängig zu übersetzen und erst mit dem Linker zusammenzufügen.

Teile, die unabhängig voneinander übersetzt werden können, werden **Module** genannt.

Modul

C geht sogar noch einen Schritt weiter. Ein Modul, der nur Unterprogramme enthält, wird in zwei Quelldateien gespeichert.

- Eine Datei enthält nur die **DeklARATIONen** der Größen, die der Modul **exportiert**, also in der Regel Typdeklarationen und Prototypen von Funktionen, gelegentlich auch Konstanten und Variablen.

Solch eine Datei wird als **Headerdatei** bezeichnet.

- Die andere Datei enthält die **Definitionen** der zu exportierenden Größen, also insbesondere die Definitionen der Prototypen aus der Headerdatei.

Diese Datei kann aber auch zusätzliche lokale Funktionen realisieren, die dann nur innerhalb des Moduls verwendet werden können.

Auf die Headerdatei greift sie mit einer `include`-Direktive zu.

Üblicherweise erhalten Headerdatei und Implementation eines Moduls **denselben Namen**, bei der Headerdatei um `.h` und bei der Implementation um `.c` erweitert.

Will man auf einen separaten Modul zugreifen, muss man zwei Dinge tun.

- Dem Compiler muss man im **rufenden** Modul die Größen des **gerufenen** Moduls bekanntmachen.

Das geschieht mit einer `include`-Direktive für die Headerdatei des gerufenen Moduls.

- Dem Linker müssen die Module, die er zusammenbinden soll, einzeln genannt werden.

Beispiel, Headerdatei bin.h

```
1  /**-----
2  * checks for a binary digits; that is 0 or 1
3  *-----*/
4  int isbdigit(int ch);
5
6  /**-----
7  * convert the string in str, interpreted as a
8  * sequence of binary digits, to an integer
9  *-----*/
10 int btoi(char str[]);
```

Beispiel, Implementation bin.c

```
1  #include "bin.h"
2
3  //-----
4  int isbdigit(int ch) {
5      if (ch == '0' || ch == '1')
6          return 1;
7      return 0;
8  }
9
10 //-----
11 int btoi(char str[]) {
12     int x = 0;
13     for(int i = 0; str[i]; i++) {
14         if (!isbdigit(str[i]))
15             return -1;
16         x *= 2;
17         if (str[i] == '1')
18             x++;
19     }
20     return x;
21 }
```

Beispiel, Testprogramm bin-demo.c

```
1  #include <stdio.h>
2  #include "bin.h"
3  //-----
4  void usage(char str[]) {
5      printf("USAGE: %s <binary-number>\n", str);
6      return;
7  }
8  //=====
9  int main(int argc, char* argv[]) {
10     if (argc != 2) {
11         usage(argv[0]);
12         return -1;
13     }
14     int x = btoi(argv[1]);
15     if (x < 0) {
16         usage(argv[0]);
17         return -1;
18     }
19     printf("dec: %d\n", x);
20     printf("oct: %o\n", x);
21     printf("hex: %x\n", x);
22
23     return 0;
24 }
```

Separate Compilation (1/2)

Wie übersetzt und bindet man Programme, die aus mehreren Modulen bestehen?

```
gcc -o bin-demo bin-demo.c bin.c
```

Konnten beide Module nacheinander fehlerfrei übersetzt werden, schließt sich der Lauf des Linkers unmittelbar an, so dass wir das ausführbare Programm erhalten.

Stellt der Compiler einen Fehler fest, etwa im Modul `bin-demo.c`, so übersetzt er zwar auch noch `bin.c` (oder versucht es zumindest); der Linker wird jedoch nicht mehr gestartet.

Der nächste Schritt ist nun natürlich, die Fehler zu beseitigen. Danach kann man dann neu übersetzen.

Für größere Projekte ist dieses Vorgehen ungeeignet. Bei jedem Compileraufruf zehn oder mehr Module hinschreiben ist umständlich, ganz davon abgesehen, dass es bei so vielen Modulen nicht besonders effizient ist, alle immer neu zu übersetzen.

Separate Compilation (2/2)

Günstiger ist es, die Objektdateien aufzubewahren und nur dann neu zu erzeugen, wenn sie nicht mehr auf dem aktuellen Stand sind.

Ein Modul muss neu

- wenn man an seiner Implementation etwas geändert hat.
- wenn an einer der Headerdateien, die er benutzt, eine Änderung vorgenommen wurde

Zum Beispiel könnte die Parameterliste einer Funktion verändert worden sein. Statt sich nun den Kopf darüber zu zerbrechen, wo die Funktion überall aufgerufen wird, lässt man den Compiler arbeiten. Er wird (wenn er dem Standard entspricht) jede Abweichung der Argumente eines Aufrufs von der (geänderten) Parameterliste bemerken.

Objektdateien erzeugen mit dem Compiler.

```
gcc -c bin.c  
gcc -c bin-demo.c
```

Binden mit dem Linker.

```
gcc -o bin-demo bin-demo.o bin.o
```
